

Plataforma Educativa

CMS educativo interactivo para infantil y primaria

Manual Tecnico y de Usuario - V-0.1.0

Este documento describe la arquitectura, los endpoints de la API, el modelo de datos y las instrucciones de instalacion y uso de la plataforma educativa en su version inicial (V-0.1.0). Esta dirigido tanto al equipo tecnico (administradores de sistema, desarrolladores) como a los editores de contenido.

Fecha: 2026-06-13

Version del software: V-0.1.0

Licencia: MIT (open source)

PARTE I - MANUAL TECNICO

1. Vision general del sistema

La Plataforma Educativa es un CMS web diseñado para publicar y ejecutar ejercicios interactivos (HTML/CSS/JS) y articulos de texto dirigidos a alumnado de infantil y primaria en Espana. El acceso al catalogo es publico, sin registro. Dos roles privilegiados - admin y editor - gestionan los contenidos desde una API REST.

Objetivos principales

- Publicar y ejecutar ejercicios educativos de forma segura para menores.
- Permitir a editores gestionar contenidos (CRUD, versionado, papelera) con baja friccion.
- Ofrecer a familias y alumnos descubrir contenido por catalogo y buscador.
- Sostenibilidad economica sin comprometer la privacidad (DSA/RGPD).
- Mantenibilidad por una sola persona: simplicidad por encima de la sofisticacion.

Restricciones de diseno clave

- Sin cuentas de alumno. Sin cookies de seguimiento. Visitas anonimas y agregadas.
- Publicidad/afiliacion solo en zonas de adultos (DSA art. 28).
- Ejercicios aislados en iframe sandbox desde subdominio separado.
- SQLite WAL como unica base de datos. Sin microservicios ni broker.

2. Arquitectura

Estilo arquitectonico

Monolito modular con Arquitectura Hexagonal (Ports & Adapters) y patrones de Clean Architecture. DDD tactico ligero con contextos acotados como modulos. CQRS solo logico (separacion lectura/escritura, sin segunda base de datos).

Regla de dependencia

Las dependencias siempre apuntan hacia dentro: infrastructure -> application -> domain. El dominio no conoce FastAPI, SQLAlchemy, ni Pydantic de request. Jamas.

Contextos acotados (V-0.1.0 implementados)

identity	Autenticacion JWT + gestion de usuarios (admin/editor)
content	CRUD de contenidos educativos, versionado e historial, papelera

Contextos acotados (planificados)

taxonomy	Ciclos, cursos, asignaturas - catalogos configurables
media	Ficheros adjuntos (imagenes, audio)
auditing	Registro de auditoria de acciones
analytics	Conteo de visitas anonimas y agregadas
configuration	Nombre del sitio, logotipo, configuracion global

3. Estructura de carpetas

```
plataforma-educativa/  
  backend/  
    app/  
      bootstrap.py      # wiring ORM (side-effect imports)  
      config.py         # Settings via pydantic-settings  
      main.py          # FastAPI app, routers, exception handlers  
      contexts/  
        identity/  
          domain/      # model, ports, services (puro Python)  
          application/  # commands, queries, dtos, handlers  
          infrastructure/ # ORM models, repos, auth_service  
          api/         # router, schemas, dependencies  
          content/     # misma estructura  
        shared/  
          domain/base.py # Entity, DomainError, DomainEvent  
          infrastructure/ # database.py, unit_of_work.py  
      migrations/      # Alembic  
      tests/  
        unit/  
        integration/  
      pyproject.toml  
  frontend/           # React + TypeScript (pendiente)  
  docker-compose.yml  
  CLAUDE.md  
  CHANGELOG.md  
  docs/
```

4. Instalacion y despliegue

Requisitos del servidor

- Docker Engine 24+ y Docker Compose v2
- Puerto 8000 accesible (o 80/443 con proxy inverso Nginx/Caddy)
- Subdominio sandbox.<dominio> apuntando al mismo servidor (ejercicios interactivos)
- Al menos 512 MB RAM, 1 vCPU

Despliegue rapido con Docker Compose

```
# 1. Descomprimir el zip de distribucion  
unzip plataforma-educativa-v0.1.0.zip  
cd plataforma-educativa-v0.1.0  
  
# 2. Copiar y configurar variables de entorno  
cp .env.example .env  
nano .env    # editar SECRET_KEY y demas valores  
  
# 3. Levantar servicios  
docker compose up -d  
  
# 4. Ejecutar migraciones  
docker compose exec backend alembic upgrade head
```

5. Verificar

curl http://localhost:8000/health

-> {"status": "ok"}

Variables de entorno

DATABASE_URL	sqlite:///data/plataforma.db (por defecto)
SECRET_KEY	Clave secreta para JWT - OBLIGATORIO cambiar en produccion
ACCESS_TOKEN_EXPIRE_MINUTES	Duracion del token JWT (default: 60)
MEDIA_BASE_DIR	Ruta base para almacenamiento de ficheros HTML (default: ./media)
ALLOWED_ORIGINS	Origenes CORS permitidos (default: http://localhost:5173)
ENVIRONMENT	development production

Crear el primer usuario administrador

En V-0.1.0 no existe interfaz de administracion web. El primer usuario admin se crea llamando directamente al endpoint POST /api/v1/users/. Para arrancar el sistema desde cero use el script de semilla incluido:

```
docker compose exec backend python -m app.scripts.seed_admin \
  --email admin@ejemplo.com --password TuPasswordSegura123
```

5. Referencia de la API (V-0.1.0)

La API sigue principios REST con prefijo /api/v1. La autentificacion utiliza el flujo OAuth2 Password Bearer (JWT HS256). Los errores devuelven JSON con campo 'detail' y codigo HTTP estandar.

Contexto identity - Autenticacion y usuarios

Metodo	Ruta	Auth	Descripcion
POST	/api/v1/auth/token	-	Obtener token JWT (form: username + password)
GET	/api/v1/users/	admin	Listar todos los usuarios
POST	/api/v1/users/	admin	Crear nuevo usuario (admin o editor)

Contexto content - Catalogo publico (sin autentificacion)

Metodo	Ruta	Auth	Descripcion
GET	/api/v1/contenidos/	-	Listar contenidos publicados
GET	/api/v1/contenidos/{id}	-	Obtener contenido por ID

Contexto content - Gestion de contenidos (editor+)

Metodo	Ruta	Auth	Descripcion
POST	/api/v1/contenidos/	editor	Crear nuevo contenido (borrador)
PUT	/api/v1/contenidos/{id}	editor	Actualizar titulo, descripcion, cuerpo
POST	/api/v1/contenidos/{id}/publicar	editor	Publicar contenido
POST	/api/v1/contenidos/{id}/archivar	editor	Retirar de publicacion
DELETE	/api/v1/contenidos/{id}	editor	Mover a papelera (soft-delete)
POST	/api/v1/contenidos/{id}/restaurar	editor	Restaurar desde papelera

Contexto content - Panel admin

Metodo	Ruta	Auth	Descripcion
GET	/api/v1/admin/contenidos/	admin	Listar todos los contenidos (incl. papelera)
GET	/api/v1/admin/contenidos/papelera	admin	
DELETE	/api/v1/admin/contenidos/{id}/purgar	admin	Borrado permanente

6. Modelo de datos

Tabla: user

id	TEXT PK - UUID en formato string
email	TEXT UNIQUE NOT NULL
password_hash	TEXT NOT NULL - hash Argon2id
role	TEXT NOT NULL - 'admin' 'editor'
is_active	INTEGER NOT NULL - 1=activo, 0=inactive
created_at	TEXT NOT NULL - ISO 8601 UTC

Tabla: content

id	TEXT PK - UUID
tipo	TEXT NOT NULL - 'interactivo' 'texto'
titulo	TEXT NOT NULL
descripcion	TEXT
autor_id	TEXT FK -> user.id
ciclo_id / curso_id / asignatura_id	TEXT - IDs de taxonomia (nullable)
idioma	TEXT DEFAULT 'es'
tags_json	TEXT - JSON array de etiquetas
is_published	INTEGER - 1=publicado
is_deleted	INTEGER - 1=en papelera (soft delete)
body_html	TEXT - cuerpo HTML (tipo=texto; sanitizado)
hash_html	TEXT - SHA-256 del fichero HTML (tipo=interactivo)
created_at / updated_at	TEXT - ISO 8601 UTC

Tabla: content_version (historial inmutable)

id	TEXT PK - UUID
content_id	TEXT FK -> content.id
version_no	INTEGER - numero correlativo (1, 2, 3...)
metadata_snapshot_json	TEXT - JSON con titulo, descripcion, etiquetas, tipo, idioma
body_html	TEXT - cuerpo en este momento
hash_html	TEXT - hash del HTML interactivo
created_by	TEXT FK -> user.id
created_at	TEXT - ISO 8601 UTC

Restriccion UNIQUE (content_id, version_no) garantiza unicidad del historial. Las versiones nunca se borran; restaurar un contenido no destruye versiones anteriores.

7. Modelo de seguridad

Autenticacion y autorizacion

- Contraseñas hasheadas con Argon2id (parametros por defecto de argon2-cffi).
- JWT HS256 firmado con SECRET_KEY. Expiracion configurable (default 60 min).
- Roles: admin (gestion completa + usuarios) y editor (solo contenidos).
- Endpoints de lectura publica sin autenticacion.

Aislamiento del sandbox (ejercicios interactivos)

Los ejercicios HTML/JS nunca se sirven desde el origen de la aplicacion. Se sirven exclusivamente desde sandbox.<dominio> con:

- iframe con atributo sandbox='allow-scripts' (sin allow-same-origin).
- CSP estricta en las cabeceras del subdominio sandbox.
- El JS del ejercicio no puede acceder a cookies ni localStorage del origen principal.

Sanitizacion asimetrica

- HTML de articulos (tipo=texto): SIEMPRE sanitizado en servidor y cliente.
- HTML de ejercicios (tipo=interactivo): NUNCA sanitizado (debe ejecutar JS). Por eso se sirven aislados.

Privacidad y menores

- Sin cuentas de alumno. Sin perfilado. Sin cookies de seguimiento.
- Visitas contadas de forma anonima y agregada (en batches, no por peticion).
- Publicidad/afiliacion prohibida en zonas de contenido para menores (DSA art. 28).

8. Tests (V-0.1.0)

Tests unitarios	27 tests - dominio identity + content, handlers con MagicMock
Tests de integracion	19 tests - endpoints REST con SQLite en memoria (StaticPool)
Total	46 tests - todos pasan en V-0.1.0

Ejecutar los tests

```
cd backend
pip install -e '[dev]'
pytest                # todos los tests
pytest -v tests/unit/  # solo unitarios
pytest -v tests/integration/ # solo integracion
ruff check .           # linting
black --check .        # formato
mypy app               # tipos
```

PARTE II - MANUAL DE USUARIO

9. Primeros pasos para el editor

Obtener un token de acceso

Toda operacion de gestion requiere un token JWT. Para obtenerlo, envia una peticion al endpoint de autenticacion con tu email y contraseña:

```
curl -X POST http://tu-dominio.com/api/v1/auth/token \
  -d 'username=editor@ejemplo.com&password=TuPassword' \
  -H 'Content-Type: application/x-www-form-urlencoded'
```

Respuesta:

```
# {"access_token": "eyJ...", "token_type": "bearer"}
```

Guarda el valor de access_token. Incluyelo en la cabecera Authorization de todas las peticiones de gestion:

```
Authorization: Bearer eyJ...
```

10. Gestion de contenidos

Crear un contenido (borrador)

Un contenido nuevo se crea siempre como borrador (no publicado).

```
curl -X POST http://tu-dominio.com/api/v1/contenidos/ \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "titulo": "Las vocales",
    "descripcion": "Ejercicio interactivo para repasar las vocales",
    "tipo": "interactivo",
    "idioma": "es",
    "etiquetas": ["lengua", "vocales"],
    "body_html": null
  }'
```

Tipos de contenido

interactivo	Ejercicio HTML/CSS/JS autocontenido. Se ejecuta en el cliente, aislado en iframe. El campo body_html es null; el HTML se sube como fichero y se referencia por hash SHA-256.
texto	Articulo o recurso en HTML enriquecido. El contenido va en el campo body_html y siempre se sanitiza antes de almacenarse y mostrarse.

Ciclo de vida de un contenido

- BORRADOR - recién creado. Visible solo para editores y admins.
- PUBLICADO - visible en el catalogo publico. POST .../{id}/publicar
- ARCHIVADO - retirado del catalogo sin borrar. POST .../{id}/archivar
- PAPELERA - borrado logico. Solo visible en el panel admin. DELETE .../{id}

- RESTAURADO - recuperado de la papelera. POST .../{id}/restaurar

Publicar un contenido

```
curl -X POST http://tu-dominio.com/api/v1/contenidos/{id}/publicar \
-H "Authorization: Bearer $TOKEN"
```

Archivar (retirar de publicacion)

```
curl -X POST http://tu-dominio.com/api/v1/contenidos/{id}/archivar \
-H "Authorization: Bearer $TOKEN"
```

Mover a la papelera (soft-delete)

```
curl -X DELETE http://tu-dominio.com/api/v1/contenidos/{id} \
-H "Authorization: Bearer $TOKEN"
```

Restaurar desde la papelera

```
curl -X POST http://tu-dominio.com/api/v1/contenidos/{id}/restaurar \
-H "Authorization: Bearer $TOKEN"
```

Historial de versiones

Cada vez que se modifica un contenido se crea automaticamente una version inmutable. Las versiones registran titulo, descripcion, etiquetas, idioma, tipo, cuerpo HTML y hash del fichero interactivo en el momento del cambio. Las versiones nunca se borran y restaurar no destruye el historial.

11. Gestion de usuarios (solo admin)

Listar usuarios

```
curl http://tu-dominio.com/api/v1/users/ \
-H "Authorization: Bearer $ADMIN_TOKEN"
```

Crear un nuevo usuario

```
curl -X POST http://tu-dominio.com/api/v1/users/ \
-H "Authorization: Bearer $ADMIN_TOKEN" \
-H "Content-Type: application/json" \
-d '{"email": "editor@ejemplo.com", "password": "Pass1234!", "rol": "editor"}'
```

- rol puede ser 'admin' o 'editor'.
- El email debe ser unico en el sistema.
- La contrasena se hashea con Argon2id; nunca se almacena en texto plano.

12. Catalogo publico (sin autenticacion)

Cualquier visitante puede consultar el catalogo y ver los contenidos publicados sin necesidad de autenticarse ni registrarse.

Listar contenidos publicados

```
curl http://tu-dominio.com/api/v1/contenidos/
```

Obtener un contenido especifico

```
curl http://tu-dominio.com/api/v1/contenidos/{id}
```


La respuesta incluye título, descripción, tipo, idioma, etiquetas y, para tipo=texto, el HTML del cuerpo. Para tipo=interactivo, la respuesta incluye la URL del iframe sandbox que el navegador puede mostrar directamente.

13. Roadmap (proximas versiones)

V-1.1.x	Contexto taxonomy (ciclos, cursos, asignaturas configurables)
V-1.2.x	Frontend React - catalogo publico + panel de editor
V-1.3.x	Contexto media (imagenes y ficheros adjuntos)
V-1.4.x	Contexto auditing + analytics (visitas anonimas y agregadas)
V-1.5.x	Contexto configuration (nombre del sitio, logotipo)
V-2.0.0	Busqueda FTS5, E2E con Playwright, zona de adultos con anuncios